

Data & AI Governance Framework on Databricks Lakehouse

Current architecture · How the structure makes new regulations a config change · UK integration walkthrough

The accelerator started as a DPDP-India compliance POC. A recent refactor turned it into a **regulation-adaptive governance platform on the lakehouse**: the core code knows nothing about any specific regulation, and every regulation-specific value lives in a single directory. Adding a new regulation is a configuration change, not a rewrite.

This doc shows what the architecture looks like today, why it's genuinely modular, and walks through a concrete UK-data-protection integration to demonstrate the claim.

Current architecture

Four layers. Layers 1 and 2 are regulation-agnostic code; Layer 3 is regulation-specific configuration; Layer 4 is the user-facing skin.

LAYER 1 — PLATFORM CORE (REGULATION-AGNOSTIC)

Lakehouse + Unity Catalog substrate

Medallion pipeline, column-mask UDFs, row-filter mechanism, audit log, Delta Sharing, consent-event schema, DSR plumbing, Agent Bricks integration, persona orchestrator. Byte-identical across deployments.



LAYER 2 — GOVERNANCE TAXONOMY (SEMI-UNIVERSAL)

Universal PII patterns, rights catalogue, pack loader

11 universal PII patterns (email, phone, card, name, address, DOB, IP, medical, insurance, CVV, bank account), the `PIIPattern` dataclass, the 9-entry `RIGHT_CATALOGUE` superset, and the typed-accessor pack loader.



LAYER 3 — REGULATION PACK (SPECIFIC, SWAPPABLE)

One directory per regulation

Everything regulation-specific: compliance rules, rights activations, notice bodies, retention defaults, residency allow-list, breach SLA, language registry, region-specific PII patterns. Selected by a single environment variable.



LAYER 4 — PERSONA & DELIVERY (REGULATION-AWARE SKIN)

Dashboards, Genie spaces, agent prompts

Four persona roles (CCO / GC / CMO / CFO) constant across regulations. Their dashboard content reads from pack-populated views, so swapping a pack re-skins the dashboards automatically.

Directory map

Where each layer physically lives in the repo. The `regulations/` directory is the swap point: DPDP India sits there today; a UK pack would sit as a sibling, same shape, loaded by the same env var.

```
repo root
├── governance_core/
│   ├── pack_loader.py
│   ├── rights.py
│   ├── consent_model.py
│   └── pii_patterns/
│       └── universal.py
│
├── regulations/
│   ├── dpdp_2023/
│   │   ├── pack.yaml
│   │   ├── rules.yaml
│   │   ├── rights.yaml
│   │   ├── notices.yaml
│   │   ├── retention_defaults.yaml
│   │   ├── residency.yaml
│   │   ├── breach_sla.yaml
│   │   ├── languages.yaml
│   │   └── pii_patterns.py
│   └── uk_gdpr/
│       ├── pack.yaml
│       ├── rules.yaml
│       ├── rights.yaml
│       ├── notices.yaml
│       ├── retention_defaults.yaml
│       ├── residency.yaml
│       ├── breach_sla.yaml
│       ├── languages.yaml
│       └── pii_patterns.py
│
├── pipelines/
│   ├── medallion.py
│   ├── classification_dlt.py
│   ├── phase1_bootstrap.py
│   └── retention_enforcement.py
│
├── schemas/
│   └── pii_patterns.py
│
├── scripts/
│   ├── setup_all_personas.py
│   ├── apply_pii_masks.py
│   ├── apply_persona_row_filters.py
│   ├── dsr_discovery.py
│   ├── dsr_erasure.py
│   └── generate_multilang_notices.py
│
└── dashboards/
    └── personas/
```

*← Layer 1/2 – regulation-agnostic code
reads REGULATION_PACK env var
9-entry RIGHT_CATALOGUE superset
consent-event schema contract
11 universal PIIPatterns*

← Layer 3 – swap point

— active today (India)

*— future pack (lands here, same 9 files)
jurisdiction=GB, ICO, £17.5M ceiling
UK GDPR articles + DPA 2018 sections
7 rights activated
en-GB body
UK + adequacy list
72h to ICO
NHS, NINO, UK postcode, UTR*

*← Layer 1 – no edits needed for new regulations
Bronze → Silver DLT
PII classifier (uses active pack patterns)
reads the pack, populates compliance tables*

*← Layer 1 – no edits needed
composition shim (universal + active pack)*

*← Layer 1 – no edits needed
persona orchestrator (regulation-agnostic)*

*reads pack for reason / locale / SLA
reads pack.languages()*

*← Layer 4 – pack-aware skin
CCO / GC / CMO / CFO (constant across regs)*

Two observations that make the modularity visible:

- Only **regulations/** **grows** when a new regulation is added. The UK pack is 9 files next to the 9 DPDP files — no file under any other directory changes.
- The **active pack is selected at bootstrap time** by one env var (**REGULATION_PACK=dmdp_2023** today, **uk_gdpr** once that pack is authored). Code elsewhere reads from the active pack via the loader; it never references a regulation by name.

What's modular and what's regulation-specific

The modularity claim rests on one invariant: **no regulation-specific value appears as a literal in any file under **pipelines/** , **scripts/** , or **schemas/**** . All such values go through the pack loader.

IN THE CORE (REGULATION-AGNOSTIC)	IN THE PACK (ONE FILE PER CONCERN)
• Medallion pipeline (Bronze → Silver) • DLT classification (universal patterns) • Column masks • Row filters • UC grants (persona layer) • Audit log queries • Delta Sharing • DSR plumbing (discovery + erasure) • Retention enforcement • Persona orchestrator • Universal PII patterns + rights catalogue • Pack loader (typed accessors)	• pack.yaml — jurisdiction, authority, penalties • rules.yaml — compliance rules + citations • rights.yaml — activated rights + SLAs • notices.yaml — notice bodies + consent purposes • retention_defaults.yaml — per-purpose retention • residency.yaml — allowed countries + filter targets • breach_sla.yaml — notification deadlines • languages.yaml — locale registry • pii_patterns.py — region-specific patterns

The pack loader exposes typed accessors — consumers don't read yaml directly:

```
from governance_core.pack_loader import active_pack

pack = active_pack()
for rule in pack.rules(): ...
retention_days = pack.retention_default(purpose="marketing_email")
allowed = pack.residency_allowed_countries()
```

Switching regulations is a two-touch operation — author a pack directory, set one env var. Nothing under **pipelines/** , **scripts/** , or **schemas/** changes.

Integrating the UK data protection regime (walkthrough)

Concrete demonstration. The POC currently ships DPDP-India only; the example below is what a UK pack would look like when that phase begins. It shows the *entire* authoring surface — nothing outside this directory would change.

Directory layout

```
regulations/
├─ dpdp_2023/           ← already shipped (India)
└─ uk_gdpr/            ← authored when the UK phase begins
    ├─ pack.yaml        jurisdiction=GB, ICO, £17.5M ceiling
    ├─ rules.yaml       rules keyed to DPA 2018 + UK GDPR articles
    ├─ rights.yaml      7 rights activated
    ├─ notices.yaml     en-GB body, consent legal basis
    ├─ retention_defaults.yaml  per-purpose retention (storage limitation)
    ├─ residency.yaml   UK + adequacy list
    ├─ breach_sla.yaml  72h to ICO + high-risk subject notification
    ├─ languages.yaml   en-GB primary, cy-GB / gd-GB optional
    └─ pii_patterns.py  NHS number, NINO, UK postcode, UTR
```

Representative pack contents

`pack.yaml` :

```
name: "UK Data Protection Act 2018 (as amended by UK GDPR 2021)"
code: uk_gdpr
jurisdiction: GB
supervising_authority: "Information Commissioner's Office (ICO)"
primary_locale: en-GB
max_penalty:
  - { amount: 17_500_000, currency: GBP, trigger: "higher-tier infringement" }
  - { amount: 8_700_000, currency: GBP, trigger: "standard infringement" }
activates:
  rights: [access, rectification, erasure, portability, objection, restriction, no_auto_decision]
  pii_pattern_packs: [universal, uk]
  minor_age: 13
  consent_default: opt_in
```

`pii_patterns.py` (UK-specific):

```
from governance_core.pii_patterns.universal import PIIPattern, ...

NHS_NUMBER = PIIPattern(
    pattern_id="nhs_number", category=CATEGORY_HEALTH, sensitivity=SENSITIVITY_CRITICAL,
    regex_pattern=r"\b\d{3}[\s-]?d{3}[\s-]?d{4}\b",
    column_hints=["nhs", "nhs_number", "patient_nhs"], ...)

NINO = PIIPattern(
    pattern_id="uk_nino", category=CATEGORY_DIRECT_GOV, sensitivity=SENSITIVITY_CRITICAL,
    regex_pattern=r"\b[A-CEGHJ-PR-TW-Z]{2}\d{6}[A-D]\b", ...)

UK_POSTCODE = PIIPattern(...)
UTR = PIIPattern(...) # Unique Taxpayer Reference

IN_SPECIFIC_PATTERNS = [NHS_NUMBER, NINO, UK_POSTCODE, UTR]
```

The other files (`rules.yaml` citing UK GDPR articles, `rights.yaml` activating 7 rights with SLAs, `residency.yaml` listing the UK + adequacy countries, etc.) follow the same straightforward yaml shape.

Activation

```
export REGULATION_PACK=uk_gdpr
databricks bundle run phase1_bootstrap --target dev
```

What the deploy then produces

TABLE / VIEW	BEFORE (DPDP)	AFTER (UK)
<code>bronze.compliance_rules</code>	Rules citing DPDP Sec. 5, 8, 11, 16, 33	Rules citing UK GDPR Art. 5 / 17 / 32 + DPA 2018
<code>compliance.notice_versions</code>	3 seeded + 7 machine-translated Indian languages	en-GB primary + any locales declared in <code>languages.yaml</code>
<code>consent_events_log.retention_duration_days</code>	Per-purpose DPDP values	Per-purpose UK values (storage-limitation)
<code>compliance.residency_filter</code> UDF body	<code>country IN ('India')</code>	<code>country IN ('United Kingdom', ...)</code>
<code>silver.pii_findings</code>	Includes Aadhaar, PAN, IFSC	Includes NHS Number, NINO, UK Postcode
DSR erasure audit trail	<code>en-IN</code> locale + <code>"DPDP Sec. 12(b) right to erasure"</code>	<code>en-GB</code> locale + <code>"UK GDPR Art. 17 right to erasure"</code>

What does NOT change

- Zero edits to `pipelines/`, `scripts/`, or `schemas/`.
- Zero edits to `governance_core/` — universal patterns + pack loader are regulation-agnostic.
- Zero edits to persona dashboards or Genie spaces — they read from pack-populated views.
- Existing integration test suites continue to pass — they assert structural invariants that survive pack swaps.

Effort

The yaml-editing portion is roughly 1–3 working days for a contributor who understands the regulation. The larger exercise is legal review of the notice body and rule citations — a domain-expert task, outside the framework's scope. The framework removes the engineering bottleneck; the legal review remains human work per pack.

The claim made concrete: adding a second regulation is authoring 9 files inside `regulations/uk_gdpr/` — it is not a refactor, not a parallel build, not a fork. The pack directory is the complete authoring surface.

Why Unity Catalog specifically

Every governance primitive the accelerator uses is UC-native. A customer gets a turnkey UC governance posture; the pack is the wrapper for their regulatory context.

GOVERNANCE CONTROL	UC-NATIVE	FRAMEWORK BENEFIT
Column tags	Built-in	PII classification writes directly to UC — lineage, searchability, Catalog Explorer discovery come free
Column masks + row filters	Built-in	Tokenisation + jurisdiction scoping at query time, enforced on every UI path (SQL editor, Genie, REST, notebook, dashboard)
Table / view grants	Built-in	Persona layer is 100% UC grants — no app-tier RBAC to maintain
Audit log (<code>system.access.audit</code>)	Built-in	Record-keeping obligations across regulations answered by one query
Lineage	Built-in	DSR discovery walks the lineage graph without customisation
Delta Sharing	Built-in	External auditor access via read-only shares — no accounts to provision
AI Functions + serving endpoints	UC-registered	Classification + multilang notice generation run inside the workspace boundary